Distributed Voting System with Edge–Cloud Architecture and Fault Tolerance using GCP

Modern distributed systems are composed of multiple independent computing nodes that communicate over a network to achieve a shared objective. Unlike traditional centralized applications, these systems must operate under conditions of partial failure, network latency, and inconsistent connectivity.

In real-world scenarios such as elections, surveys, and large-scale analytics systems, data is continuously generated at the edge (user devices or distributed nodes) and processed in the cloud. This introduces key challenges in consistency, fault tolerance, and coordination across distributed components.

In this laboratory activity, students will work in **groups** to design and implement a **Distributed Voting System using Google Cloud Platform (GCP)**. Each group will simulate multiple edge nodes generating votes independently, while cloud services handle ingestion, messaging, processing, and storage. The system will then be evaluated under both normal and failure conditions to analyze its reliability and behavior as a distributed system.

By the end of this laboratory activity, students should be able to:

1. Design an event-driven distributed system using cloud services
2. Implement edge-to-cloud data pipelines using HTTP and Pub/Sub
3. Apply fault tolerance techniques such as retries and idempotency
4. Analyze system behavior under failure and recovery conditions
5. Evaluate trade-offs between consistency, latency, and scalability

What is Cloud Computing, Edge Computing, and Fault Tolerance?

Cloud computing provides on-demand access to scalable computing resources such as virtual machines, serverless services, and managed databases. These resources are

abstracted from physical infrastructure and dynamically allocated based on demand, enabling scalable and elastic system design.

Edge computing refers to performing computation closer to the data source. Instead of sending all data directly to the cloud, edge nodes perform local generation or batching of data before transmission. *This reduces latency and distributes computation closer to users or devices.*

Fault tolerance is the ability of a system to continue operating correctly even when parts of the system fail. In distributed systems, failures are expected and may include node crashes, message duplication, network delays, or service interruptions. Fault-tolerant systems use techniques such as retries, idempotency, and message queues to ensure correctness despite these failures.

To solidify these concepts, you will apply them by building a distributed voting system.

Main Activity

You will design a distributed voting system using GCP where multiple edge nodes generate votes and cloud services process and aggregate them. The system must remain functional even when failures occur in either edge or cloud components.

The architecture follows an event-driven pipeline:

Edge Nodes → Cloud Run API → Pub/Sub → Worker Service → Firestore

Each group is expected to implement, deploy, and test each layer while ensuring correct system behavior under both normal and failure conditions. The expected output of the system is a continuously updated collection in Firestore where each document represents a processed vote, uniquely identified per user and poll.

GCP SETUP AND SYSTEM CONFIGURATION

Before implementing the distributed voting system, each group must first prepare a working environment in Google Cloud Platform (GCP). This setup ensures that all system components such as API, messaging, processing, and storage, can communicate properly within a single cloud project.

Step 1: Create GCP Project

Begin by accessing the Google Cloud Console at https://console.cloud.google.com

Create a new project that will contain all resources for your distributed system. Use the following naming format to clearly identify your group:

```
cs323-voting-system-groupX
```

After creating the project, make sure it is selected as the active project in the top navigation bar. All subsequent configurations must be performed within this project. This project will act as the container for all services used in your system, including Cloud Run, Pub/Sub, and Firestore.

Step 2: Enable Required Services

Once the project is active, the required cloud services must be enabled. These services provide the infrastructure needed for your distributed system.

Navigate to **APIs & Services** → **Library**, then enable the following:

- Cloud Run API, which will be used to deploy the ingestion and worker services
- Pub/Sub API, which will handle asynchronous message communication
- Firestore API, which will serve as the system's persistent storage

These services form the core architecture of your system, so ensure that all of them are successfully enabled before proceeding.

Step 3: Create Firestore Database

Next, set up the database that will store all processed votes.

Go to **Firestore Database** and click **Create Database**. When prompted, select **Native Mode**, as this supports real-time scalable applications.

Choose a region that is geographically close to your location. If available, select:

```
Asia-Southeast1
```

This reduces latency and improves system responsiveness during testing. Firestore will act as the final storage layer where processed votes are written by the worker service.

Step 4: Create Pub/Sub System

After setting up the database, configure the messaging system that connects the API and worker components.

Navigate to **Pub/Sub** → **Topics** and create a new topic named:

```
vote-topic
```

This topic will act as the central message channel where incoming votes are published.

Next, create a subscription for this topic. Name it:

```
vote-sub
```

Set the subscription type to **Pull**, which allows the worker service to retrieve messages asynchronously. This Pub/Sub configuration is critical, as it enables decoupled communication between system components and supports fault-tolerant message processing.

EDGE COMPUTING (CLIENT NODES)

After completing the cloud infrastructure setup, the next step is to simulate **edge computing behavior**. In this system, each group member will act as an independent edge node responsible for generating and sending vote data to the cloud.

Edge nodes represent user devices or distributed clients in a real-world system. They operate independently, which means they may produce data at different speeds, experience delays, or temporarily go offline. This variability is intentional, as it reflects real distributed system behavior.

Step 1: Understanding Edge Node Responsibility

Each edge node must generate vote data containing a user identifier, poll identifier, selected choice, and timestamp. These votes will then be transmitted to the Cloud Run API using HTTP requests.

Before implementation, ensure that each group member runs their own instance of the edge node script to simulate distributed sources of data.

Step 2: Implementing Vote Generation

The system begins by generating synthetic vote data. Each vote must be unique to simulate real user interactions.

Students are given a scaffolded function where the core structure is already provided, but must be extended to simulate multiple edge nodes.

```
def generate_vote():
    """
    Extend this function to simulate multiple edge nodes.
    Add an edge_id or node identifier to distinguish sources.
    """
    return {
        "user_id": str(uuid.uuid4()),
        "poll_id": "poll_1",
```

```
        "choice": random.choice(["A", "B", "C"]),
        "timestamp": time.time()
    }
```

At this stage, the focus is on ensuring that each node behaves independently rather than producing identical synchronized output.

Step 3: Sending Data to the Cloud

Once votes are generated, they must be sent to the Cloud Run API. This simulates communication between edge and cloud layers. Students must implement the request logic and improve it with retry handling to simulate unreliable network conditions.

```
def send_vote(vote):
    """
    Implement retry logic here.
    Simulate network instability by handling request failures.
    """
    try:
        requests.post(API_URL, json=vote)
    except Exception as e:
        print("Transmission failed:", e)
```

The goal is not only successful transmission but also resilience under failure conditions.

Step 4: Simulating Real Edge Behavior

To reflect real distributed systems, edge nodes must not operate in a perfectly predictable pattern. Instead, they should simulate variability in execution.

Each node should introduce random delays and continuous execution loops to mimic real-world user activity.

```
def run_edge_node():
    """
    Add randomness to simulate real-world edge behavior.
```

```
    Consider intermittent delays or pauses in execution.
    """
    while True:
        vote = generate_vote()
        send_vote(vote)
        time.sleep(random.uniform(1, 3))
```

Each group member should run this script independently to create multiple concurrent edge sources.

CLOUD INGESTION LAYER (API)

Once edge nodes are active, the system transitions to the cloud ingestion layer. This layer is responsible for receiving incoming vote requests and forwarding them into the messaging system.

The Cloud Run API acts as the entry point of the distributed system. It must remain lightweight, fast, and stateless, ensuring that it can handle multiple incoming requests without blocking or processing heavy logic.

Step 1: Receiving Edge Requests

The API receives vote data via HTTP POST requests. Each request must be validated before being processed further.

```
@app.route("/vote", methods=["POST"])
def receive_vote():
    vote = request.get_json()

    """
    Validate incoming vote:
    Ensure user_id, poll_id, and choice exist.
    """

    if not vote:
        return {"error": "Invalid payload"}, 400
```

At this stage, the API is only responsible for validation, not storage or computation.

Step 2: Publishing to Pub/Sub

After validation, the vote must be forwarded to Pub/Sub. This ensures decoupling between ingestion and processing layers. Students must complete the publishing logic using the Pub/Sub client.

```
    try:
        """
        Publish the validated vote to vote-topic.
        This enables asynchronous processing.
        """
        return {"status": "accepted"}, 200

     except Exception as e:
        return {"error": str(e)}, 500
```

The key design principle here is **non-blocking ingestion**, where the API does not wait for processing results.

DISTRIBUTED PROCESSING LAYER (WORKER)

The worker service represents the processing layer of the distributed system. Unlike the API, which handles incoming requests, the worker is responsible for consuming messages, processing them, and storing results. This layer operates asynchronously and continuously listens for new messages from Pub/Sub.

Step 1: Receiving Messages from Pub/Sub

The worker subscribes to the Pub/Sub subscription and processes each incoming message individually.

```
def process_vote(message):
    """
```

```
    Decode and parse incoming vote message.
    Ensure safe handling of malformed data.
    """
    vote = json.loads(message.data.decode("utf-8"))
```

Each message represents a single vote that must be processed reliably.

Step 2: Ensuring Data Consistency (Idempotency)

In distributed systems, duplicate messages may occur. To prevent inconsistent data, each vote must be uniquely identified before storage.

```
doc_id = f"{vote['user_id']}_{vote['poll_id']}"
```

This enforces **idempotent writes**, ensuring that duplicate message deliveries result in the same final state rather than multiple records.

Step 3: Storing Processed Votes

Once validated and checked for duplicates, the vote is stored in Firestore.

```
db.collection("votes").document(doc_id).set(vote)
```

Firestore acts as the final persistent storage layer of the system.

Step 4: Acknowledging or Retrying Messages

After processing, the worker must confirm message handling. If successful, the message is acknowledged; otherwise, it is retried.

```
message.ack()
```

If an error occurs, the message is not acknowledged, allowing Pub/Sub to retry delivery.

Step 5: Continuous Processing Loop

The worker runs continuously, listening for new messages without manual intervention. This reflects the nature of real-world distributed processing systems, where services remain active and event-driven.

FAULT INJECTION AND SYSTEM BEHAVIOR ANALYSIS

At this stage, your distributed voting system is already fully deployed across edge nodes, Cloud Run, Pub/Sub, and Firestore. The system should now be operating under normal conditions, with votes continuously flowing from edge nodes to the cloud.

In this phase, you will intentionally introduce controlled failures and abnormal behaviors to evaluate how the system behaves under distributed system conditions. This involves both modifying your Python edge node code and adjusting configuration settings in Google Cloud Platform (GCP).

Step 1: Simulating Message Duplication at the Edge Layer

Begin by modifying the edge node so that the same vote is sent multiple times before generating a new one. This simulates retry behavior or network-induced duplication in distributed systems.

Inside your edge node script, adjust the sending logic as follows:

```
vote = generate_vote()

send_vote(vote)
send_vote(vote)  # intentional duplicate transmission
```

Or alternatively, you may simulate repeated delivery using a loop:

```
for i in range(3):
    send_vote(vote)
```

After applying this change, allow the system to continue running normally. You should observe an increase in message volume flowing into Pub/Sub and potentially into Firestore depending on whether deduplication is implemented.

Step 2: Simulating Worker Failure Using Cloud Run Configuration

Next, simulate a failure in the processing layer by disabling the worker service in Cloud Run. Go to your **Cloud Run worker service**, click **Edit & Deploy New Revision**, and modify the scaling configuration:

```
Minimum instances = 0
Maximum instances = 0
```

This ensures that no worker instance is actively running, effectively simulating a backend failure. While the worker is disabled, **keep your edge nodes running continuously** so that vote data continues to be generated and sent to the system:

```
while True:
    vote = generate_vote()
    send_vote(vote)
    time.sleep(random.uniform(1, 3))
```

During this phase, observe that:

- Cloud Run API continues to accept requests
- Pub/Sub continues buffering incoming messages
- Firestore temporarily stops receiving updates
- No system-wide failure occurs despite worker downtime

This demonstrates how distributed systems isolate failure to specific components.

Step 3: Restoring Worker Service and Observing Recovery

After observing system behavior during failure, restore the worker by redeploying or updating the Cloud Run service.

Set configuration back to:

```
Minimum instances = 1 (or default)
```

Once restored, the worker will automatically reconnect to Pub/Sub and begin processing queued messages without manual intervention.

Observe that:

- Previously unprocessed votes are delivered in batches
- Firestore updates resume automatically
- No manual replay or recovery logic is required

This demonstrates **asynchronous recovery through message persistence**.

Step 4: System Behavior Monitoring and Debugging Counters

To better observe system behavior, you may optionally add simple debugging counters in both edge and worker layers. These are not required for functionality but are useful for tracing system flow.

Edge node (vote tracking):

```
print(f"Vote generated: {vote['user_id']} | Choice:
{vote['choice']}")
```

Worker service (processing confirmation):

```
print(f"Processed vote: {vote['user_id']} | Poll:
{vote['poll_id']}")
```

These logs allow you to verify:

- Whether all generated votes reach the worker

- Whether duplicates appear during processing
- Whether recovery reprocesses queued messages correctly

SYSTEM EVALUATION AND PERFORMANCE ANALYSIS

After completing fault injection and recovery testing in Part 5, your distributed voting system has now been exposed to normal operation, message duplication, worker failure, and recovery scenarios. In this phase, you will evaluate how the system behaves across these conditions using actual observable data from GCP and your program logs.

Unlike traditional software evaluation, distributed system evaluation is based on behavior across multiple components rather than a single execution result. You will compare how data moves through edge nodes, Cloud Run, Pub/Sub, and Firestore under different conditions.

Step 1: Measuring End-to-End Latency (Edge to Firestore)

Begin by measuring how long it takes for a vote generated at the edge to appear in Firestore.

To do this, use the timestamp already included in your vote payload and compare it with the time the vote appears in Firestore. You may use your existing vote structure and optionally enhance visibility by printing timestamps at both ends:

```
"time_created": time.time()
```

In the worker service, you may add:

```
print(f"Received at worker: {vote['user_id']} | Time:
{time.time()}")
```

Then compare with Firestore timestamp to estimate delay. During observation, take multiple samples (at least 10 votes) and note the variation in latency under different system conditions.

Step 2: Evaluating System Throughput (Processing Capacity)

Next, evaluate how many votes the system can process over a period of time. You will use observable data from three layers:

- Edge logs (votes generated)
- Pub/Sub metrics (messages queued and delivered)
- Firestore records (final stored votes)

To support tracking, you may optionally include simple counters:

```
print("Vote sent:", vote["user_id"])
```

and;

```
print("Vote processed:", vote["user_id"])
```

Observe how throughput behaves under:

- normal execution
- worker downtime (queue accumulation)
- recovery phase (batch processing)

Step 3: Evaluating Fault Tolerance Behavior

Now compare system behavior under the failure scenarios introduced in Part 5. You will focus on three execution states:

- **Normal Operation** - The system runs with all components active. Votes flow continuously from edge nodes to Firestore with minimal delay.

- **Failure State (Worker Disabled)** - The worker is inactive, but edge nodes continue sending data. Pub/Sub buffers incoming messages while Firestore updates stop.
- **Recovery State -** The worker is restored and begins processing queued messages automatically.

During observation, focus on the following:

- whether any votes were lost during worker downtime
- whether duplicate votes appear in Firestore
- whether the system recovers automatically without manual intervention

Step 4: Evaluating Consistency Across Components

Next, verify whether the system produces a consistent final state after execution.

Compare total counts across:

- number of votes generated at edge nodes
- number of messages observed in Pub/Sub
- number of documents stored in Firestore

These values should be approximately equal unless duplicates were intentionally introduced.

You should also check whether the final dataset in Firestore reflects all valid votes after recovery. This step evaluates whether the system maintains **eventual consistency**, meaning that even if intermediate states differ, the final state converges correctly.

Step 5: System Trade-Off Observation

Finally, analyze the trade-offs observed during system execution. Focus on how design decisions affected system behavior:

- Pub/Sub improves reliability but introduces delay due to buffering
- Cloud Run scaling improves flexibility but introduces cold-start behavior

- Edge computing reduces server load but increases distribution complexity
- Idempotency (if implemented) improves correctness but adds processing overhead

During this step, you are not expected to modify the system, only to interpret its behavior based on what you observed.

REFLECTION AND ANALYSIS

All observations, analysis, and insights must be documented in your README.md file. Each student must write an **individual reflection in paragraph form**, based on their actual experience while implementing and testing the distributed voting system on Google Cloud Platform.

The reflection should focus on how the system behaved across its different components (edge nodes, Cloud Run API, Pub/Sub, worker service, and Firestore), particularly under normal operation, increased load, and failure-recovery scenarios.

Each student may discuss insights related to the following aspects:

- Differences observed between **sequential execution and distributed execution**, especially in how vote generation, transmission, and processing are coordinated
- System performance behavior as the number of votes increases, particularly in terms of **latency, queue buildup in Pub/Sub, and processing throughput**
- Challenges encountered during implementation and deployment, including **GCP configuration, service integration, debugging distributed components, and handling asynchronous execution**
- Insights on system behavior under distribution, such as **communication overhead, message buffering, synchronization between worker and Firestore, and eventual consistency**
- Situations where distributed execution improved performance, as well as cases where it introduced unnecessary complexity, delay, or debugging difficulty

These aspects are provided as **guides only and do not need to be followed exhaustively**. Students are free to structure their reflection as long as the discussion remains relevant to the system behavior and distributed computing concepts observed during the activity.

Reflections must be grounded in actual implementation experience and system behavior observed during execution, not theoretical explanations. Students are expected to connect their insights to real outcomes from edge nodes, Cloud Run processing, Pub/Sub messaging, and Firestore updates.

SUBMISSION GUIDELINES

Each group must submit a complete GitHub repository containing all components of the distributed voting system. The repository must be organized in a clear structure, separating edge node scripts, Cloud Run API, worker services, and supporting configuration files.

The repository must include a README.md file containing:

- System overview and architecture explanation
- Step-by-step setup and execution instructions for running the system on GCP
- Individual student reflections (as described in the Reflection and Analysis section)

A system architecture diagram must also be included. This diagram should clearly illustrate the flow of data from edge nodes to Cloud Run, through Pub/Sub, and into Firestore, reflecting the actual implemented system.

Instead of static screenshots, each group must submit a **short demonstration GIF or video** showing the system in operation. This should clearly demonstrate:

- Vote generation at edge nodes
- API ingestion via Cloud Run
- Message flow through Pub/Sub
- Worker processing
- Firestore updates in real time

Finally, each group must include the **deployed Cloud Run API endpoint URL** as proof of successful deployment and system accessibility.

Optional enhancements such as dashboards, monitoring tools, or visualization interfaces may also be included to demonstrate deeper system understanding.